

CANopy project draft

Lorand Kedves, 2026-04-09, v1.0

Origins

The Artificial Intelligence National Laboratory ([MILAB](#), funded by [HUN-REN SZTAKI](#), an institute of the [Hungarian Academy of Sciences](#)) was an umbrella organization hosting projects to **provide oversight, extract knowledge and allow better interaction with data-rich information systems**, in different fields. My research focused on analyzing data in financial reports collected by the EU ESEF (since 2023) and US EDGAR (since 2008), but we shared experiences and results. As I had worked for Continental Automotive for two years, I found the **Dashboard interfaces – from buttons to a context-oriented touch screen** workshop by [AHJK](#) (Audi Hungaria Faculty of Vehicle Engineering) particularly interesting.

Using screens is inevitable because a modern car has much more features than a decade ago. Assigning physical controls to each of them would make the dashboard like an airplane cockpit: very expensive to build and inconvenient to use. People are familiar with touch screen interfaces and expect the automotive industry to follow these trends, especially because of leaders like Tesla.

However, there are serious drawbacks of this approach.

- **Adaptation:** Drivers can get used to physical controls, interact with the most common systems automatically, basically muscle memory. Some blocks are the same in different models of the same brand (ventilation, audio) and similar even among most of the cars (lights, turn indicators).
- **Coherence:** The cost of building physical controls forced manufacturers to spend more time with experimenting, collecting feedback and improving them. The software is easy to write and change, but this does not give as deep analysis of the individual tasks and the whole driver-car interaction.
- **Efficiency:** The menu system is complicated and does not follow the individual Pareto-optimum of the drivers. They use their own 20% of features in the 80% of time, find it inconvenient to make multiple steps to access them.
- **Reliability:** The screen must follow a wide range of ambient light conditions; too bright or dim display is inconvenient or even dangerous. The lack of physical feedback (the edges of the button or knob, the feel of pressing or turning it), failure of registering the action (press, slide, pinch) also cause problems, especially in challenging driving conditions like bumpy roads or steep turns.
- **Eyes on the road:** The current screen-oriented interaction requires more attention of the driver than the physical controls. In a study the driver spent twice the times looking at controls instead at the road when they appeared on a screen and not as physical buttons while executing the same set of standard tasks.
- **Extreme scenarios:** Similarly, it is dangerous to increase the cognitive load of the driver in a critical traffic scenario, when the reaction speed is key to avoid, or at least decrease the severity of an accident.

These factors resulted some unexpected pushback against the screen-oriented movement.

- There are many **user complaints** about the interfaces, from inconvenience to slow or unreliable operation. The requests are **often contradicting**, making a general solution hard.
- **Safety regulators require creating physical controls** to the critical or most often used services to keep the drivers' eyes on the road. It is hard to design such interfaces when you must keep their state in sync with the virtual controls on the screen.
- The **manufacturers often can't realize the cost reduction** expected from software that replaces physical controls, while the whole process becomes less predictable ("move fast and break things" does not work here).

There is a significant gap between expectations and the actual results of integrating informatics into the automotive industry, and this is an opportunity for a new approach. The **CANopy project uses only existing technologies but in a different combination** to display data and control features. It lays out a **multi-stage development process** towards a more organic smart vehicle interaction.

The North Star Vision

North Star is a **long-term (5-10 years) vision**, assuming a successful integration with the global automotive industry, proper hardware development in collaboration with major automotive brands, without major drawbacks outside the scope of the current analysis.

However, the following **detailed plan contains multiple stages** with much less risks and external dependencies, **with identified customer groups and viable products**.

Screen

The CANopy project focuses to support the driver with **one, non-touch, movable e-ink display** close to the edge of the windscreen.

The screen is on a movable arm like computer monitors, mounted between the steering wheel and the center of the dashboard. The driver can move the screen around, from directly above the steering wheel to the center where the touch screens are traditionally placed today. **The screen is for output only; drivers can move it to a position most convenient to see it.** They do not have to reach it, **no need for any compromise.**

The e-ink technology gives more comfort than any other technology. Imagine yourself driving in a sunny day in the mountains, going in and out of tunnels. Your car's screens constantly change their brightness, yet still hard to see the numbers and indicators, while your partner besides you can read an eBook all time with no problem. The reason is that **"active" displays use light to display information** and compete with the sometimes suddenly changing external conditions. On the other hand, **"passive" ones use darkness: decrease the reflection of the ambient light.** You don't need a shade or (and) extra strong backlight, **your eyes adapt to the changes** in daylight, and you switch on a tiny lamp or backlight at night.

Controller

We accept that it is impossible to assign physical buttons, knobs, etc. to all features that a modern car offers, regardless of their convenience – but similarly, using touch screens to control them comes with inconvenient compromises.

A different compromise is having a limited set of **physical buttons and knobs but change their meaning depending on the actual context** displayed on a screen.

Another aspect of this analogy is that most of the comfort features do not depend on the brand. All cars have window and door controls, ventilation, audio, navigation assist, seat heating and movement, etc. although drivers and passengers must learn where the manufacturer put the controls in their vehicle.

There is another industry that met and successfully solved this problem. You have a TV set, an amplifier, CD/DVD player, set top box, etc. in the living room but often only **one universal remote control**. You see the current context of the TV, DVD player or set top box on the big TV screen, instead of individual panels on all devices. To navigate and change values, you need the up – down – left – right direction buttons, next item / next page, previous item / previous page, ... buttons, number pad and the standard color menu selectors, etc. **You find it natural that these controls interact with different parameters of different devices because you have visual feedback on the screen.**

In the car, you have the display in your preferred position to keep the road in your peripheral vision. You also have **a similar remote control, locked to your seat in the most convenient position** for your hand. You don't have to look down to it, **your fingers find the buttons** to activate a menu, increase or decrease a selected value with the convenient physical “click” feeling, **even in a turn or on a rough road**. With standard mounting brackets on both sides of the driver seat, you can attach your own 3D printed controller customized to your hand for deeper adaptation support.

Integration

The fundamental question is, how this screen and controller can interact with the car features.

With acceptable simplification, you should be aware that **cars built in the past 25+ years are complex information systems where sensors and controllers** interact for efficiency and safety. In an average modern car, there are 100+ ECUs (Electronic Control Units), each **connected to the CAN** (Controller Area Network) bus and talk with each other over it. When mechanics want to learn about the condition of your car, they connect their computer to a CAN endpoint under your steering wheel to let the diagnostics software talk with these ECUs.

The **CANopy mounts connect to the CAN bus at the physical attachment point**, while the inner connector would be a USB (micro-A or C) socket or cable that you plug into the physical Screen or Controller devices. The “passive” Screen reads data from the CAN bus as the different ECUs talk with each other and displays the selected values according to the currently selected context. The Controller is daisy-chained to the Screen via a CAN bus extension and sends custom CAN messages to it. Some of them are handled by the Screen itself (selecting panels and data items), others are passed through the CAN bus to reach the target ECUs.

There is a technical inconvenience that the different ECUs in various cars “may talk a different language to say the same thing”. Therefore, CANopy uses two sets of data: the “abstract car” with the general features you want to display or control, and the “language” of the actual car. To use CANopy in your car, you will download its language and the translation rules between it and the abstract car from our global repository (one of the mentioned “product stages”). However, CANopy describes both “languages” the same way, therefore it is possible to display and control any feature unique to a brand or model (and even share it over the CANopy portal) without the general translation layer.

Customization / Extension

The Screen listens into the CAN communication, selects the data to display on the selected panel and updates the visualization. Naturally, the layout and visual parameters of the panels are configurable (as natural in general mobile platform development), and these configurations can be combined. The Controller is yet another ECU from this perspective, therefore, the connection between controller buttons and actions are also configurable.

The result is that ***drivers can customize the way they want to see various parameters of their cars***, from the simple speed and fuel gauges to oil temperatures or various engine sensors. They can use the CANopy portal to share these configurations with other drivers, who may not have the resources to do the same but can pick a convenient one to use on their screen or controller. If the configuration uses only the “abstract car” level, this configuration is stored in the Screen and can be activated on another car (to the level that supports the abstract features). ***Drivers can transfer their “driving experience” between cars*** – this feature may be important for car rental companies or organizations with drivers and different car models.

Manufacturers can also publish “official” Screen / Controller configurations for their models. Beyond the default, they can add deeper views into the technical components for enthusiasts, or others that resemble older models for making a transition easier. With a modular setup, they can support the public to extend or partially use their configurations. This is also a business opportunity with a granular subscription model.

The CANopy model relies on configurations and isolated “sandboxed” plugins, no free coding for data connection and transformation, this reduces the risks of malicious activities. Also, as the Screen has ultimate control over what can physically sent to the car CAN bus, ***manufacturers can provide a gatekeeper configuration that rejects invalid or dangerous commands***.

Although the CAN frame standards evolved and will change over time, the general structure and hardware is basically the same with limited number of variants. This means, CANopy can be used in any car that has a CAN bus – most of all running on the roads today. ***The project scope can be extended later to other segments that also use CAN***, like heavy vehicles (trucks, buses, agricultural machines), ships and even industrial automation that use the J1939, NMEA 2000 and CANopen standard, respectively.

Technical Analysis

The main source of this analysis is [CSS Electronics](#) portal. This company focuses on making hardware CAN tools and build custom solutions using an existing, mostly open-source software stack. I have checked some referred software projects on GitHub. The main implementation language is Python. The [asammfd](#) library seems to be the de facto standard tool to read CAN data files and there are some, mostly platform-dependent solutions in other languages. The overall feeling is that ***there are tactical solutions and working business models, but no true strategic attempt.***

Accessing useful data from the CAN bus is complicated. The manufacturer picks the 100+ ECUs from different manufacturers, then combine their output to provide control services, monitoring, visualization, etc. modules. They generally do not publish the applied protocols, which would be necessary to understand everything that happens in the car. Enthusiasts try to reverse engineer these protocols from the ECU description and monitoring the CAN communication and share their results, but this is not a reliable source.

The situation is a bit better with heavy vehicles, the J1939 is a globally adopted standard, its role is like the previously mentioned “abstract car”: each vehicle may have some custom ECUs with special data, but the generally needed part is covered by J1939. CSS sells a J1939 variant on a per-user license.

The general need of homogeneous CAN data access created the OBD (On Board Diagnostics) standard. This is the “CAN Esperanto” that each vehicle should speak so that mechanics can get information from their components. Although there are multiple variants (US and EU differences for cars, another over the J1939 standard, etc.), it is still a more reliable source as the manufacturer must declare which of the few variants the car is compatible, then CANopy can use that protocol.

Over time, CAN got saturated, and the strict priority scheme (like engine messages overtake window movement commands) can introduce inconvenient delays. For this reason, ***manufacturers separated lower priority communication*** (entertainment, comfort, etc. features) ***to one or more Local Interconnect Networks (LIN)***. Although LIN networks connect to the CAN via a LIN gateway, the LIN communication does not appear as CAN messages, talking to LIN nodes requires knowing the communication protocol over the gateways. This is viable only in collaboration with the manufacturer.

Connecting to the CAN bus requires a very reliable hardware, preferably certified by the manufacturer. There are many existing (often tightly coupled hardware-software) solutions, even selecting one requires expertise, avoiding too deep dependency would also be important.

Screen manufacturing is less problematic, but can also be avoided: if the CAN communication can appear on a device with a USB connector, a mobile application can take over the role of the Screen, either using a general Android / iOS phone / tablet as a display, or relaying the display role to the car’s own integrated display via the generic Android Auto or CarPlay – or custom API in collaboration with the manufacturer (similar to embedded, branded navigation systems).

The controller is a relatively simple hardware, assuming that the CAN bus protection is solved by the initial connector.

Milestones

The conclusion of the technical analysis is that the environment is challenging. Direct CAN access requires a complex and safe hardware requiring strong knowledge on the field. There are different CAN frame formats, all of them are binary. The meaning of any frame is defined externally in DBC files, the syntax is still complex but at least text. Accessing these files is complicated, manufacturers do not publish them. With the introduction of LIN segments, CAN does not cover all areas. The “North Star Vision” requires collaboration with manufacturers or at least, established partners – they need working prototypes with exciting features to join.

1 – Read existing DBC and CAN files

The first step is to apply my existing knowledge management platform, Dust / TAP, create a DBC reader to read the frame definitions, then an MF4 file for actual CAN frames. This is a pure programming task, open-source samples are on GitHub test data is available from CSS Electronics; their implementations can be used to validate the results. The assumption is that TAP would allow very fast, in-memory CAN frame management with minimal effort. If this assumption fails, the project can be closed with minimal cost.

The possible buyers / partners are companies like CSS Electronics who provide services to fleet managers and alike. However, they already invested a lot into their existing solutions, the chance of having customers at this point is quite low.

2 – Support CAN expert mechanics

He has proper knowledge in verifying and describing a CAN analyzer, wrote 100+ Python scripts and use many existing tools. He also has tools to collect CAN data from cars, so without own hardware, we get live test data, can collect requirements and feedback. The assumption is that our tool should be more useful and faster than the existing ones, allow writing test scripts within TAP.

The product is a tool that CAN mechanics would like to use instead of their current tools. At this point we should investigate the business model of a portal that could become a public repository of DBC files, or even allow mechanics share their scripts either for free or for some fee.

3 – Reach CAN enthusiasts

There is a larger group who spend a lot of time and effort tuning their own cars. They often have hardware to connect CAN and display information or even interact with the ECUs. At this level, we still rely on the customer to collect CAN data, but now directly in the car while driving. We should implement the Screen as a mobile application and perhaps, the Controller as well; these people have the incentive to design and share interface definitions, we could get great feedback from them.

This target group is much larger than CAN mechanics, also spend a lot of money on their cars, the assumption is that with them CANopy should break even and start making profit. Also, the portal with interface and monitor configurations should become popular and get the interest of the final target groups, manufacturers and experts.

4 – Collaborate with experts

Experts are those companies who officially extend cars with CAN-related services, provide online monitoring or offline analytics for fleet managers, etc. They already reach all data that they need and have all tools. The assumption is that our TAP-based approach allows cost reduction on their side when they want to implement new services or adapt to changes in standards. This is optional but can be a significant milestone if at least one company decides to work with us.

5 – Collaborate with manufacturers

This is the final stage. Up to this point we assumed that the partner gets all information about the CAN communication that is needed for the required services. However, a manufacturer can provide direct access to the full CAN communication, including the LIN segments. This would allow CANopy to become the primary interface to the complete feature set of a car.

The benefit for the manufacturer can be:

- Cost reduction, if building the modular, configuration-based user interface is less expensive and more robust than their current, often criticized method.
- Better service to the general customer, officially supported Screen variants, CANopy mount points and accessories in the packages.
- Deeper connection to the CAN enthusiasts and mechanics through properly managed knowledge sharing, perhaps over the CANopy portal.
- Selling new services to previous models with minimal extra cost: provide modern features to a 10, 20 years old car (it only requires the DBC file and mappings).

References

- [CSS Electronics](#) portal with a lot of information and existing services, good source for finding business opportunities. They offer free test data and consultation. They focus on hardware and created a software toolchain from existing external services. They may find the CANopy software stack interesting.
- [CSS Electronics YouTube playlist](#) introducing the CAN environment.
- Videos from Can enthusiasts on different levels – showing the dedication and freedom they have, and the various parallel “tactical” software and hardware solutions. Some of these videos have over millions of views, showing interest from those who would not actually do these modifications.
<https://youtu.be/2bv3WjtGgYM> , <https://youtu.be/cSEwpu8Ofjg> ,
https://youtu.be/5lIdSN_dZIM , <https://youtu.be/SVn9uYfEQrA> ,
https://youtu.be/t_w6BzN3ac0
- [Asammfd](#) – the standard Python library. The sources (on GitHub) are huge, like almost 13000 lines of code to read the CAN mdf4 files. The alternative solution must be much smaller, otherwise the CANopy project is pointless.
- [Opendbc](#) – a Python library also on GitHub to read DBC files, together with shared DBC files. Another “tactical” solution for enthusiasts, less useful for consumers or manufacturers.